

目标一 前缀缓存实验对照表与差异解释

一 实验结果对照

实验	请求数	Prompt tokens	Cached tokens	缓存命中率	Actual Prefill tokens	TTFT p50(s)	TTFT p95(s)	TPOT p50(s)	TPOT p95(s)	Latency p95(s)
Shared Prefix	32	67584	65599	97.06%	1985	0.0397	0.1792	0.00265	0.00319	0.2653
Dispersed Prefix	32	67584	0	0%	67584	0.4642	0.7904	0.01152	0.01913	0.9524

二 公式与计算

缓存命中率 = $\Sigma \text{cached_tokens} \div \Sigma \text{prompt_tokens}$ 。共享前缀：65599 ÷ 67,584 = 97.06%；分散前缀：0 ÷ 67,584 = 0%。

实际 Prefill token 数 = $\Sigma(\text{prompt_tokens} - \text{cached_tokens})$ 。共享前缀：67,584 - 65599 = 1985；分散前缀：67,584 - 0 = 67,584。

因果自注意力：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

三 差异解释

共享前缀组。 32 条请求前 2048 个 token 完全相同且位置一致。首个请求完成预热后，这段前缀对应的 h_i 、 K_i 、 V_i 已写入 Radix Cache；后续请求在相同位置得到相同的 K/V，因此可以直接复用。每条请求只需重新 Prefill 64 个独立后缀，32×64=2,048；记录中的 4,096 表明还包含首条请求建立缓存时的 2,048 个前缀计算。

分散前缀组。 首个 token 各不相同，Radix Cache 无法找到可复用的共同前缀。即使后续 token 可能相似，因果注意力中当前位置的表示依赖此前全部可见 token；前缀一开始就不同，后续 K/V 也不能安全地当作同一段缓存复用，因此 32 条请求都执行完整 2,112 token 的 Prefill，共 67,584 token。

相同 token 与位置的前缀为什么可以复用 K/V：

相同 token 保证了 Embedding 后的向量是一样的，位置则保证 $X = X_{dec} + PE$ 时位置编码是一样的以及 Masked Self-Attention 时所接受的上下文信息是一样的

命中后仍需计算 哪些 token

未命中的后缀 token 以及 decoder 时的 token (Q)

TTFT 与 TPOT 为什么呈现不同变化

KV Cache 命中主要减少 Prefill 阶段的计算，因此 TTFT（首 token 时间）明显下降；而 TPOT（每个输出 token 的生成时间）主要发生在 Decode 阶段，每次仍需要生成新的 token 并计算新的 K/V，所以变化较小。

共享前缀的实际 Prefill 仅为分散前缀的 6.06%（4,096/67,584），减少约 93.94%，所以平均延迟从 0.8440 s 降至 0.1749 s，p95 从 0.9615 s 降至 0.3304 s。TTFT 主要包含排队和 Prefill，因而通常随缓存命中而明显下降；TPOT 属于 Decode 阶段，每步仍需基于新 token 计算并生成，且两组输出长度相同，所以变化通常小于 TTFT。

SGLang 请求处理总流程

TokenizerManager.generate_request创建一次模型生成请求。



Scheduler.event_loop_normal / waiting_queue调度器的普通事件循环 / 等待队列



get_new_batch_prefill / match_prefix_for_req获取一批需要Prefill的新请求。检查这个请求有没有已经缓存的前缀



RadixCache.match_prefix一个专门快速寻找共同前缀的缓存结构。



PrefillAdder / ScheduleBatch决定：哪些token需要真正送进模型计算



model worker: Prefill, Decode模型开始做预填充（模型计算产生KV并保存）和解码



RadixCache.cache_finished_req / cache_unfinished_req完成请求 保存完整缓存 还没完成，也可能已有部分KV。保存部分结果。



TokenizerManager: streaming output生成一点，返回一点。

目标二 请求流程说明

目标二说明 | 关键函数与文件先列出，再回答四个问题

关键函数所在文件

- entrypoints/http_server.py
 async def generate_request()
- managers/tokenizer_manager.py
 TokenizerManager.generate_request()
 _send_one_request()
 _wait_one_response()
- managers/scheduler.py
 event_loop_normal()
 process_input_requests()
 handle_generate_request()
 _add_request_to_queue()
 get_next_batch_to_run()
 get_new_batch_prefill()
- managers/schedule_policy.py
 match_prefix_for_req()
- mem_cache/radix_cache.py
 RadixCache.match_prefix()
 cache_finished_req() / cache_unfinished_req()

1 请求如何进入等待队列

TokenizerManager.generate_request() 先统一请求格式、创建请求状态并分词，然后 _send_one_request() 通过 send_to_scheduler.send_pyobj() 把已分词请求发送给 Scheduler。Scheduler 的 event_loop_normal() 接收请求，process_input_requests() 通过 TypeBasedDispatcher 分发到 handle_generate_request()，创建 Req。_add_request_to_queue() 检查队列容量、尝试 KV 预取，最后执行 waiting_queue.append(req)。

2 前缀匹配如何减少本轮 Prefill token

get_new_batch_prefill() 遍历 waiting_queue，req.init_next_round_input(tree_cache) 调用 match_prefix_for_req()，再由 RadixCache.match_prefix() 在 Radix Tree 中寻找最长公共前缀。命中的前缀已有 KV，可直接复用并跳过；init_next_round_input() 用 input_len - len(prefix_indices) 设置本轮 extend_input_len，PrefillAdder 再决定能否加入批次。SGLang 至少保留 1 个 token 做 Prefill。

3 新产生的 KV 如何写回缓存

Prefill/Decode 产生的新 KV 先保存在 KV Cache Pool。cache_unfinished_req() 或 cache_finished_req() 取得 token_ids 和 req_to_token_pool 中的 KV 位置映射，构造 RadixKey，把 values = kv_indices 传给 RadixCache.insert()。RadixCache 保存“token 前缀 → KV Cache 位置”的索引，并更新 req_to_token_pool，使后续请求能通过 match_prefix() 找到它。

4 生成结果如何流式返回

Scheduler 把输出 token 交给 DetokenizerManager，后者增量地把 token ID 转成可显示文字，再送回 TokenizerManager。_wait_one_response() 按 request id 等待新输出并 yield。HTTP server 的 stream_results() 每拿到一个结果就执行 yield b"data: " + dumps_json(out) + b"\n\n"，以 SSE 风格分块返回。

请求如何进入等待队列 ① 请求预处理与发送

TokenizerManager 处理单个请求，再通过进程通信交给 Scheduler



请求如何进入等待队列 ② Scheduler 接收与入队

普通模式主线：接收请求、按类型分发、建立 Req，再检查并加入等待队列



TypeBasedDispatcher

基于请求类型，选择对应的处理函数

它接收什么

Scheduler 接收到的对象不一定是文本生成请求，也可能是向量生成请求。分发器查看对象的类型，再决定交给谁处理。

在本流程中的位置

process_input_requests() 遍历请求，并调用 _request_dispatcher(recv_req)。生成请求随后进入 handle_generate_request()。

它决定什么

它决定调用哪个处理函数；真正创建 Req 和加入等待队列，发生在后面的处理步骤。

初始化时建立对应关系

init_request_dispatcher()

TokenizedGenerateReqInput
对应 handle_generate_request

TokenizedEmbeddingReqInput
对应 handle_embedding_request

一个具体例子

收到 TokenizedGenerateReqInput 对象，就按映射调用生成处理函数，不会把它送到 embedding 处理函数。

Req

Scheduler 用来管理一次生成任务的内部请求对象

它保存什么

把请求编号、原始文本、输入 token ID 和采样参数放在一个对象里，后面的调度与计算围绕这个对象继续。

它从哪里来

handle_generate_request() 接收已经分词的请求，再执行 Req(...), 创建内部任务记录。

它到哪里去

_add_request_to_queue(req) 处理入队条件，
waiting_queue.append(req) 保存这个对象，等待组成批次。

原文中的构造方式

```
req = Req(  
    recv_req.rid,  
    recv_req.input_text,  
    recv_req.input_ids,  
    recv_req.sampling_params,  
    ...  
)
```

四个字段分别对应

rid: 请求编号
input_text: 原始文本
input_ids: 输入 token 编号
sampling_params: 采样参数

Disaggregation

把 Prefill 与 Decode 分开安排的架构

原文的含义

Disaggregation 表示解耦或分离。这里讨论的是把 Prefill 阶段与 Decode 阶段拆开，而不是把一条句子拆成 token。

拆开的是哪些工作

Prefill 处理输入并产生相应 KV。Decode 在已有上下文基础上继续生成后续 token。两个阶段可以分别安排执行资源。

本次图使用哪个分支

笔记沿 DisaggregationMode.NULL 继续，展示普通模式下请求进入 waiting_queue 的路径。

普通模式的路径

`_add_request_to_queue(req)`

`DisaggregationMode.NULL`

检查排队容量
按条件预取缓存
加入 `waiting_queue`

阅读流程时的边界

分离模式还会有其他等待区域和处理路径。当前流程图只追踪普通模式，不把两种模式混在同一条必经链上。

HiCache 与 prefetch

关注缓存所在的层级，以及如何提前搬回可用位置

HiCache 关注的问题

已经存在的 KV 缓存可能处于不同存储层级。HiCache 关注缓存放在哪里，以及需要时怎样把它搬回来。

prefetch 的作用

预取会提前准备可能需要的缓存数据，把数据搬向后续计算更容易使用的位置，以减少真正执行时的等待。

在笔记中的入口

`_prefetch_kvcache(req)` 位于加入等待队列之前。该步骤与是否启用 HiCache、当前配置和缓存状态有关。

与 RadixCache 匹配的区别

`match_prefix()` 关注哪些 token 前缀已经有可复用的 KV 位置。prefetch 关注相应缓存数据是否需要从其他层级搬回来。

如何理解先后关系

原文提到“prefetch 发生在匹配之后”，应结合预取内部的查找步骤理解，不能据此把所有调用固定成同一顺序。入队预取与后续组批匹配应分开看。

前缀匹配如何减少 Prefill ① 查找可复用前缀

先遍历等待请求，再计算可匹配范围并查询已有缓存



PrefillAdder

根据缓存匹配结果与资源条件，决定请求能否加入本轮批次

它什么时候创建

`_get_new_batch_prefill_raw()` 中先建立 PrefillAdder。创建组件不代表请求已经进入本轮 GPU 批次。

它使用哪些信息

原文列出 `page_size`、`tree_cache` 和 `token_to_kv_pool_allocator`，并结合请求的剩余 Prefill 长度安排批次。

它实际完成的工作

请求先匹配已有前缀并计算剩余 token，再由加入器判断本轮能否接收这个请求，同时考虑分块与对齐限制。

原文中的创建方式

```
adder = PrefillAdder(  
    self.page_size,  
    self.tree_cache,  
    self.token_to_kv_pool_allocator,  
    ...  
)
```

一个示意例子

请求共 10 个 token，已有 7 个可复用前缀 token，就按剩余 3 个 token 的计算与空间需求尝试加入本轮批次。能否加入还取决于本轮资源条件。

为什么保留至少一个 token

复用已有前缀 KV 后，仍需要为继续生成执行当前输入的末尾计算

笔记中的限制

`_compute_max_prefix_len(input_len)` 限制可匹配前缀长度。在笔记描述的这条路径中，不能把整个 prompt 都作为本轮可跳过的的前缀。

保留的是哪一个 token

这里保留的是已有输入 prompt 的末尾 token，不是提前生成一个新 token。其本轮计算用于继续得到后续生成需要的输出。

为什么 KV 还不够

KV 缓存保存注意力计算要复用的中间结果；仅拿到前缀 KV 的位置，并不等于已经得到本次生成下一 token 所需的最终输出。

普通命中的示意

输入长度 10，命中前缀 7
剩余 Prefill 长度 = $10 - 7 = 3$

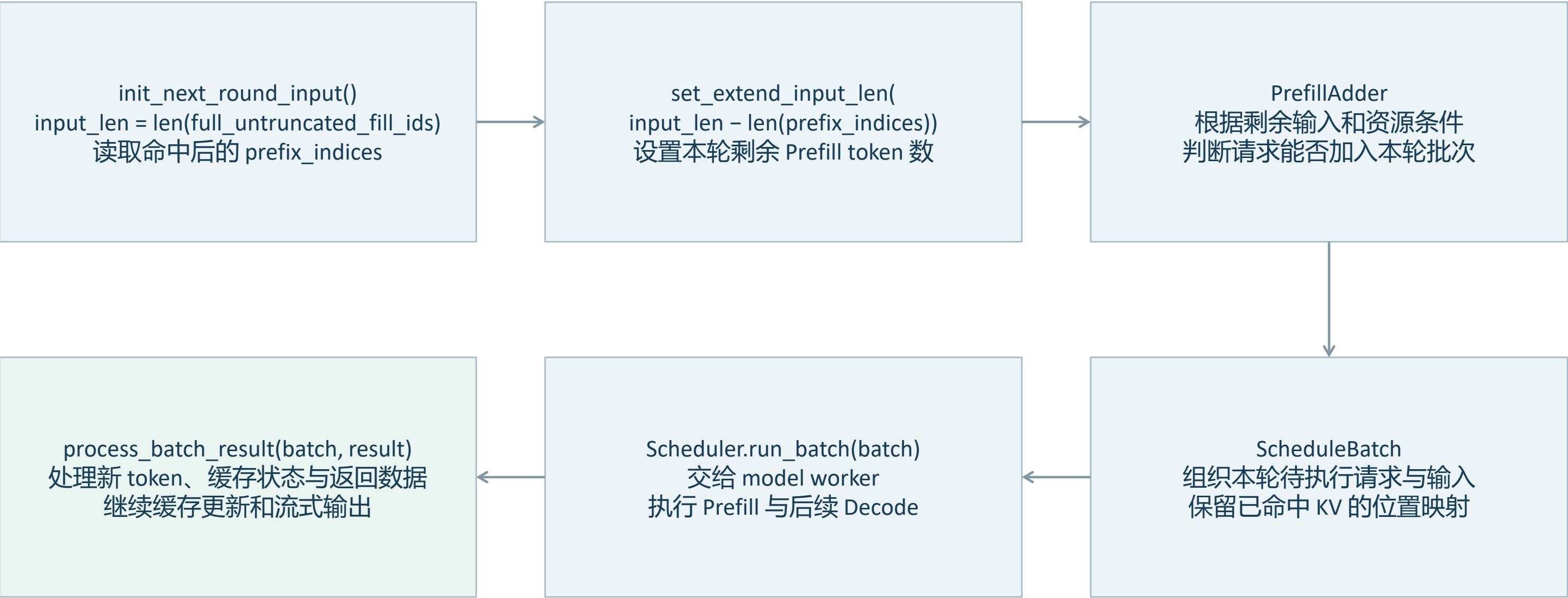
输入完全相同时的示意

按笔记中的“最多匹配 9 个”路径：
输入长度 10，可复用前缀 9
剩余 Prefill 长度 = $10 - 9 = 1$

前面 9 个位置复用 KV，最后 1 个输入 token 仍执行本轮计算。

前缀匹配如何减少 Prefill ② 组批与模型执行

匹配得到的 prefix_indices 决定可跳过的输入长度，剩余 token 才进入本轮扩展计算



例：输入 10 个 token，命中 7 个，则本轮剩余 Prefill 为 3 个；分块与对齐还会影响实际安排

新产生的 KV 如何写回缓存

先保存 KV 数据，再把 token 前缀与 KV 位置绑定到 RadixCache



生成结果如何流式返回

模型输出 token 经增量解码后回到 TokenizerManager，再逐块通过 HTTP 返回



DetokenizerManager

把生成的词元编号增量转换为可以显示的文字

为什么需要单独处理

token 与文字不是一一对应关系。一个中文字可能涉及多个 token，因此，生成一个 token 并不保证新增一个完整字符。

它怎样处理新增输出

结合已经解码的内容继续处理新 token，只把本次新增、已经可以显示的文字送出去，避免把未完整的文字片段直接交给用户。

它在返回链中的位置

Scheduler 交出输出 token，DetokenizerManager 增量解码，再把文字结果送回 TokenizerManager，供后续流式返回。

一个示意例子

第一次新增的 token 还不足以构成一个可显示字符，可以暂不增加显示文字。

后续 token 补齐这个字符后，再输出该字符。

流式输出的含义

客户端不断收到新增的文字片段，无需等完整答案生成完毕。一次片段可以包含一个或多个字符，不要求与单个 token 一一对应。

目标三模拟路由实验结果与分析

依据《目标三.docx》整理。实验使用四个独立 Python 模拟 Replica，验证前缀路由与缓存行为，未启动真实 SGLang 推理后端。D 相比 C 分散了请求，记录的端到端延迟 p95 降低，模拟缓存命中率略有下降。

一 实验指标对照

统计范围为每组 2048 条 Measurement 请求，排除 64 条 Warmup。下表将五组策略并列，R0 至 R3 表示四个模拟后端。

指标	A 默认	B10	B20 选定	C 亲和	D 混合
成功率	100%	100%	100%	100%	100%
吞吐量 请求每秒	未记录	未记录	未记录	未记录	未记录
模拟缓存命中率	99.85%	100%	100%	99.95%	99.80%
实际 Prefill token 数	N/A	N/A	N/A	N/A	N/A
TTFT p95 毫秒	N/A	N/A	N/A	N/A	N/A
端到端延迟 p95 毫秒	1.066	1.045	1.014	0.948	0.873
R0 请求数	511	486	511	2048	700
R1 请求数	483	517	500	0	450
R2 请求数	540	520	519	0	449
R3 请求数	514	525	518	0	449
请求数合计	2048	2048	2048	2048	2048

数据说明

吞吐量：原文缺少测量区间总耗时，不能用 p95 延迟的倒数计算。实际 Prefill token 数与 TTFT：模拟服务未进行模型推理，因此记为 N/A，不能填 0。

策略配置按原文记录：A 使用默认 P2C，max_ongoing_requests 为 5；B10 和 B20 保持 P2C，分别设为 10 和 20。B20 在两个候选中记录的 p95 更低，故用于后文比较。

二 问题回答

1 默认值与 B 中选定值为何表现不同

B20 的端到端延迟 p95 为 1.014 ms，低于默认 A 的 1.066 ms，下降约 4.9%；B10 为 1.045 ms。各组成功率均为 100%，请求分布也较均衡。

按原文的理解，并发上限从 5 增加到 20，允许更多请求进入副本处理流程，可能减少等待空闲名额造成的排队，所以 B20 的尾延迟略低。但增大上限不等于处理能力必然提高，副本饱和时仍可能加重排队。本次记录只能说明模拟结果的差异，尚不足以单独证明其因果关系或真实推理吞吐提升。

2 前缀亲和为什么同时带来缓存收益和热点

四个 Replica 各自维护独立缓存。C 使用 $\text{hash}(\text{prefix}) \% 4$ ，将相同前缀稳定送到同一副本，使后续请求更容易复用已有缓存，减少同一前缀在不同副本上的重复缓存建立。

代价是热门前缀会集中到固定副本。本次 C 的 2048 条请求全部落在 R0，其余三个副本没有接收测量请求，形成明显热点，增加局部排队风险。C 的模拟缓存命中率为 99.95%，高于 A 的 99.85%，但并未超过两个 B 候选的 100%。因此，缓存复用机制有收益，并不意味着本次所有对照中 C 都最高。

3 D 使用哪些信号与规则 改善什么 代价是什么

信号与规则：先使用 prefix 的哈希映射找到亲和副本，再参考各副本的负载或请求数量。亲和副本负载较低时继续发送给它，负载较高时改选最空闲副本。原文未给出完整负载更新逻辑；演示中的 request_count 持续累加，不能直接视为实时正在处理的请求数。

改善：D 的请求分布变为 700 / 450 / 449 / 449，R0 的请求占比从 C 的 100% 降到约 34.2%，热点明显缓解。端到端延迟 p95 从 0.948 ms 降到 0.873 ms，下降约 7.9%。这与分散负载、降低等待风险的解释一致。

代价：部分请求离开原本已有缓存的副本，在新副本上产生缓存未命中。模拟缓存命中率从 99.95% 降至 99.80%，下降 0.15 个百分点；同时需要维护负载信息并执行额外的路由判断。

压测端瓶颈检查

client_queue 表示客户端等待发送的队列，dispatch_lag 表示实际发送时间相对计划时间的滞后。两者仅用于确认压测端没有成为主要瓶颈，不参与路由策略优劣排名。原文未提供它们的实测数值，因此本次不能据此确认客户端瓶颈已被排除。

来源：《目标三.docx》的最终结果表、策略解释及模拟流程记录。百分比变化由表内数据计算，未补造缺失测量。